

Intent-Driven Development and Deterministic Control for AI Software Engineering

A Control-Plane Architecture Integrating IDD, Test-Driven Development, Lean Production Vault Modeling, Context Engineering, and Evidence-Based Agentic Execution

Research Thesis / Design Study

September 2026

Central proposition

Reliable autonomous software engineering requires moving durable intent, lifecycle authority, project state, context selection, and verification evidence outside the probabilistic reasoning process, while preserving AI agents as powerful but disposable reasoning workers.

Abstract

AI coding agents have shifted software engineering from a primarily code-production problem toward a control, context, and verification problem. Agents can generate substantial implementations, inspect repositories, run tools, and execute multi-step plans, yet long-running agentic work remains vulnerable to intent drift, context pollution, self-verification, stale knowledge, scope expansion, false-positive acceptance, and unrecoverable conversational state. This thesis develops a framework for addressing those weaknesses by combining Intent-Driven Development (IDD), Test-Driven Development (TDD), Lean Production Vault Modeling (LPVM), deterministic lifecycle control, project-local knowledge, role-separated execution, and evidence-bound verification.

The proposed architecture treats approved human intent as durable authority while allowing the execution plan to evolve under controlled, versioned replanning. A deterministic controller—not an AI agent—owns lifecycle transitions. Disposable Planner, Builder, Verifier, Scout, Integrator, and Promoter roles receive narrowly compiled context and capabilities. Work advances only when state-bound evidence satisfies predeclared acceptance conditions. Project knowledge remains isolated within a Git-backed project authority domain; large historical vaults are treated as untrusted source archives until curated. Human-in-the-loop Kanban gates and Obsidian visualizations provide an intelligible supervisory layer, while maintenance and break-glass paths ensure that control mechanisms themselves remain repairable.

The thesis is grounded partly in an iterative Prime/IDD engineering program in which repeated failures exposed weaknesses in apparently controlled agent workflows: contamination, provenance mismatches, incomplete acceptance checks, false-pass conditions, promotion defects, and recovery gaps. These failures are treated as design evidence rather than hidden implementation noise. The resulting hypothesis is that reliable autonomous software engineering depends less on asking increasingly capable models to police themselves and more on placing authority, state, context construction, and proof outside the reasoning process.

1. Introduction

1.1 The Shift from Code Generation to Controlled Delegation

AI coding agents increasingly act as delegated software workers rather than autocomplete systems. Contemporary intent- and spec-driven approaches explicitly respond to the gap between what a human means and what an agent infers, while context engineering focuses on deliberately curating what a model sees. These trends indicate that the limiting problem is no longer simply code synthesis; it is preserving purpose and constraints across autonomous execution.

This work begins from a practical observation: a highly capable agent can still be an unreliable custodian of its own authority. The same reasoning system that interprets requirements, edits code, chooses tests, judges success, and decides whether to continue has incentives and opportunities to collapse distinctions that conventional engineering treats as independent controls.

1.2 Research Problem

How can an AI-assisted development system preserve human intent, constrain autonomous execution, minimize context and knowledge overhead, independently verify results, recover from interruption, and still remain adaptable enough to repair its own control infrastructure?

1.3 Research Questions

- RQ1: What project state must be removed from conversational memory and made durable, explicit, and reconstructable?
- RQ2: How should human intent differ from an execution plan so that the destination remains stable while the route can evolve?
- RQ3: How can role separation and capability boundaries prevent agents from self-authorizing lifecycle transitions?
- RQ4: How can TDD and evidence binding reduce false-positive completion in agentic workflows?
- RQ5: How can context be compiled narrowly enough to reduce cost and contamination without starving workers of necessary information?
- RQ6: How can project-local knowledge grow from telemetry without allowing observations to silently become policy?
- RQ7: How can a rigid control plane remain human-repairable when the control plane itself is defective?

1.4 Contributions

The principal contribution is an integrated control architecture rather than a new language model. It combines intent authority, mutable dependency planning, context compilation, isolated execution, independent verification, evidence provenance, project-local knowledge, human gates, recovery, telemetry, and explicit control-plane maintenance into one lifecycle.

2. Background and Related Practice

2.1 Intent- and Spec-Driven Development

Current IDD and spec-driven practice emphasizes making intent explicit, reviewable, and durable before implementation. Public descriptions of IDD characterize it as an umbrella practice that structures intent and context so agents build what humans mean, while spec-driven development commonly follows a specification → implementation → validation sequence. This thesis adopts that direction but separates immutable approved intent from the mutable execution graph: intent answers what outcome is authorized; the graph answers how the system currently proposes to reach it.

2.2 Test-Driven Development as a Verification Discipline

TDD contributes more than a red-green-refactor coding rhythm. In an agentic control system, its deeper value is temporal: acceptance expectations can exist before implementation, reducing the opportunity for an implementer to redefine success after seeing its own solution. The architecture therefore generalizes TDD into evidence-first acceptance: tests, negative tests, allowed changes, artifacts, and evidence requirements are established before Builder execution wherever practical.

2.3 LPVM: Lean Context and Durable Project Memory

LPVM is treated here as a design lineage centered on reconstructability, lean prompt/context loading, durable project-local knowledge, and minimization of dependence on conversational history. The central operational idea is that the system may retain extensive knowledge, but each worker receives only the smallest sufficient slice needed for its role. This thesis extends that principle through a Context Compiler, Project Map, and explicit knowledge authority hierarchy.

2.4 Context Engineering

Context engineering can be summarized as deliberate curation of what a model sees. For coding agents this becomes an architectural concern: context has cost, authority implications, freshness, provenance, and contamination risk. A worker that searches indiscriminately through files and historical vaults spends resources and may import stale assumptions. Therefore context discovery and context use are separated.

2.5 Contemporary Agentic Workflow Patterns

Recent coding workflows increasingly use fresh-context review, isolated worktrees, dependency-aware work queues, vertical slices, and planner/implementer/reviewer separation. These patterns support the broader hypothesis that reliable agentic development benefits from process boundaries outside a single model conversation. The

proposed system adopts these useful execution patterns while adding stronger authority, evidence, recovery, and audit controls.

3. Failure Model: Why Agentic Development Drifts

Failure mode	Mechanism	Control response
Intent drift	Later prompts or local optimization weaken earlier goals	Immutable approved intent; amendments only
Context pollution	Large histories mix stale, irrelevant, or conflicting material	Compiled role context; provenance and freshness
Self-certification	Implementer chooses or interprets its own proof	Fresh independent Verifier
Scope escape	Agent explores or edits beyond task boundary	Capability envelope and project-root enforcement
False pass	Wrong/partial tests produce apparently successful evidence	Evidence evaluator binds commands, counts, hashes and provenance
Plan rigidity	Early decomposition becomes wrong as work reveals facts	Versioned mutable graph and Replan Requests
Plan mutation into scope change	Execution convenience silently changes destination	Intent/plan separation and amendment gate
Recovery loss	Chat/session death destroys operational continuity	Durable state; reconstruct from disk
Control deadlock	Rigid controller prevents its own repair	Human-only maintenance and break-glass path

4. Design Principles

The architecture is governed by a small set of invariants:

- Agents reason; the controller determines which actions are legal.
- No agent controls the lifecycle.
- Approved intent is immutable history; execution plans are mutable, versioned state.

- Execution agents receive resolved context rather than being asked to discover their working context.
- Work advances on evidence, not on an agent declaration of completion.
- Absence of proof is not success.
- Every project is a self-contained Git-backed authority domain.
- Derived indexes and visualizations may be rebuilt; they are not authority.
- Knowledge may inform policy but may not silently become policy.
- Human control must include a path to repair the control system itself.

5. Proposed Architecture

5.1 Project Authority Domain

A project begins in a clean folder and initializes Git before implementation. Project-specific intent, graph, state, evidence, telemetry, knowledge, context packets, audit history, source, tests, and documentation remain inside the project root. Sandboxes and worktrees are disposable execution boundaries; the repository is the durable boundary. External material is imported explicitly with provenance.

5.2 Intent Lifecycle

Free-form discussion precedes formal intent development. An explicit IDD transition begins structured grilling: the system summarizes inferred goals, identifies ambiguity, asks domain-aware questions, and presents unresolved decisions. No implementation authority exists during this phase. Explicit human approval creates an immutable intent version. Subsequent destination changes require an amendment referencing the prior version.

5.3 Mutable Execution Graph and Vertical Slices

A Planner decomposes approved intent into execution units (EUs) that preferentially deliver small end-to-end behaviors rather than horizontal technology layers. A separate Graph Evaluator challenges the decomposition for oversized units, missing evidence, unnecessary dependencies, coupling, and missed parallelism. The graph is versioned and may change when execution reveals new information. Replanning changes the route; amendment changes the destination.

5.4 Controller-Owned Work Queue

The controller computes executable work from authoritative graph state. An EU is eligible only when it is approved, dependencies are satisfied, required authority is available, and no human hold exists. An agent does not decide to continue merely because it believes the next step is sensible.

5.5 Role and Capability Separation

Role	Primary responsibility	Key prohibition
Conversational Prime	Discuss, explain, present state and human decisions	No silent implementation after intent approval
Planner	Propose graph and EUs	Cannot approve graph or alter intent
Graph Evaluator	Adversarially evaluate decomposition	Cannot implement
Scout / Resolver	Locate exact resources and answer context questions	No application implementation
Context Compiler	Assemble role-specific packet	No creative implementation
Builder	Produce candidate for one EU	No lifecycle, promotion, graph or intent authority
Verifier	Independently test candidate	Cannot repair candidate it judges
Integrator	Combine verified work	Cannot redesign to ease merging
Integration Verifier	Verify combined behavior	No implementation repair
Promoter	Advance accepted state	Only after required gates/evidence
Controller	Validate and execute legal transitions	Minimal creative discretion

6. Context Programming

6.1 The Context Compiler

The Context Compiler receives role, EU, authoritative state, dependency state, project map, relevant knowledge, and a budget. It emits an immutable packet containing authority, EU contract, acceptance criteria, exact dependencies, exact working resources, selected knowledge, and evidence requirements. Each item carries source, reason for inclusion, authority level, freshness, and measurable cost.

6.2 Resolved Context, Not File Hunting

Packets identify exact project-root-resolved or absolute paths, relevant symbols, purpose, and permissions. Builders should not waste execution time scanning directory trees to discover material that the control system could have resolved in advance. When information is missing, the worker issues a structured Context Request. The Scout or deterministic Project Map resolves that specific request; the Compiler decides what enters context.

6.3 Progressive Disclosure

Workers begin with the smallest sufficient packet. Additional material is granted only when a concrete need is expressed. This reduces token consumption and produces measurable context-miss telemetry. Over time, the system can learn which context bundles correlate with first-pass success without giving workers unrestricted access to the entire knowledge store.

7. Project Map and Knowledge Architecture

7.1 Project Map

The Project Map is a derived structural index connecting files, symbols, imports, tests, architecture components, EUs, requirements, verifier findings, and knowledge. Deterministic extraction is preferred over agent interpretation. The desired trace is: Intent Requirement → EU → Files/Symbols → Tests → Verification Evidence → Integration Result. Because the map is derived, corruption can be repaired by rebuilding it from repository and authoritative IDD state.

7.2 Knowledge Authority

Knowledge is separated into project truth, historical evidence, learned operational knowledge, and external/reference material. Retrieval results carry authority and freshness metadata. No old note may override current authoritative state. Telemetry-derived observations progress through Observation → Pattern → Candidate Rule → Validated Rule → Policy, with human approval at the policy boundary during early operation.

7.3 Legacy Vault Migration

Large historical Codex, Claude, Hermes, and other vaults should not be bulk-injected into new projects. They are untrusted source archives. A separate curation process inventories, deduplicates, detects stale and conflicting material, validates reusable claims, and imports only selected knowledge with provenance. This protects new projects from contamination while preserving the research value of prior work.

8. Evidence, TDD, and Independent Verification

8.1 Evidence Before Advancement

For each EU, acceptance is defined before Builder launch: behavior, required tests, negative tests where appropriate, permitted file changes, artifacts, and evidence schema. Builder completion creates a candidate, not a success state. A fresh Verifier receives an independently compiled packet and evaluates the artifact rather than inheriting the Builder's reasoning.

8.2 Evidence Binding

Evidence is bound to intent version, graph version, EU identifier, execution identifier, role, baseline Git SHA, candidate SHA, context packet hash, observed test command/result, artifact hashes, and timestamp. The evaluator rejects stale, mismatched, partial, or self-certified proof.

8.3 Negative Proof

Control-plane acceptance requires negative tests as well as positive behavior. Examples include proving that unauthorized writes are rejected, unapproved EUs cannot execute, stale evidence cannot advance state, Builders cannot promote themselves, and project workers cannot silently reach outside the project authority domain.

9. Human-in-the-Loop Control and Visualization

Early deployments intentionally favor human gates. A practical Kanban flow is Proposed → Approved → Ready → Building → Verifying → Human Review → Integrating → Integration Verify → Promoted → Done, with side states for Blocked, Needs Clarification, Replan Required, Intent Amendment Required, Failed, and Interrupted. The pause is controller-enforced, not merely requested in a prompt.

Obsidian serves as a human-facing projection of authoritative state. It can present dependency graphs, intent relationships, requirement-to-code traces, failure paths, EU evidence, timeline views, and current human gates. Operational mode answers what is happening; teaching mode explains why relationships and controls exist. Obsidian never becomes authority: human actions are validated and recorded by the controller.

10. Recovery, Maintenance, and Break-Glass Control

10.1 Disposable Workers and Durable State

Any worker process may die without invalidating the project. Durable state records active intent and graph versions, EU states, human gates, execution IDs, context hashes, worktree associations, verifier outcomes, integration status, promotion status, and unresolved findings. Restart classifies the recorded state rather than reconstructing intent from chat.

10.2 Control-Plane Maintenance

Strict control must not make the controller unrepairable. Normal Project Execution Mode forbids workers from modifying control rules. Human-authorized Maintenance Mode freezes ordinary execution, creates a dedicated repair branch/worktree, preserves the known-good controller, and requires regression tests plus human approval before activation.

10.3 Break Glass

A tiny recovery mechanism should remain as independent as practical from the main controller. A human-run shell command freezes execution, snapshots state, records the controller SHA and reason, generates a one-time nonce, and creates a repair workspace. A separate human-only approval command activates the repaired controller only after required regression and recovery proofs. Agents cannot invoke or approve this path.

11. Experimental Evidence from the Prime/IDD Program

The Prime/IDD program functions as an iterative engineering case study. It should not be represented as a completed proof of the entire architecture; rather, individual phases demonstrate particular mechanisms and failures. Early work exposed contamination, isolation, sequencing, evaluator, promotion, provenance, and recovery problems. Later rebuilds showed that apparently successful phases could still fail independent verification when acceptance contracts were stronger than the original proof.

Several lessons recur across the experiments. First, deterministic checks can themselves be incomplete, so the acceptance harness must be independently tested. Second, provenance matters: evidence is meaningful only when bound to the correct execution and baseline. Third, role labels are insufficient without enforced capabilities. Fourth, preservation of failures is essential because rewriting history destroys the evidence needed to improve the system. Finally, fresh independent testing is not redundant overhead; it is a mechanism for discovering false confidence in the control plane.

11.1 Research Status Distinction

Element	Status in this thesis
Intent approval and durable authority	Design principle supported by implementation experiments
Role separation and promotion gates	Partially demonstrated in Prime/IDD phases
Append-only lifecycle/evidence	Demonstrated in portions of the experimental program
Fresh independent verification	Repeatedly shown necessary by acceptance failures
Context Compiler / Scout	Proposed architecture to be implemented and measured
Project Map	Proposed architecture
Obsidian supervisory views	Proposed human-interface layer
Telemetry-driven context learning	Future empirical optimization
Maintenance / break-glass mechanism	Proposed safety mechanism

12. Evaluation Methodology

A credible evaluation should measure both software correctness and control correctness. The recommended protocol uses fresh projects and deliberate adversarial cases rather than only happy-path demonstrations.

12.1 Core Metrics

- Intent fidelity: proportion of accepted requirements with traceable implementation and evidence.
- False-pass rate: candidates incorrectly promoted despite missing or mismatched evidence.
- Scope violation rate: attempted and successful unauthorized reads/writes.
- Context efficiency: tokens and resources injected per successful EU; context-miss frequency.
- First-pass verification rate and retry distribution.
- Recovery success: ability to reconstruct and resume after forced process termination.
- Integration defect rate after independently verified EUs converge.
- Human intervention frequency and reason classification.

- Graph churn: replans per EU and causes of decomposition failure.
- Knowledge utility: retrieved knowledge actually referenced or associated with improved outcomes.

12.2 End-to-End Acceptance Experiment

The system should be tested from a clean folder: initialize project and Git; develop and approve intent; generate and independently critique a vertical dependency graph; stop at a human gate; compile exact context; execute a single isolated EU; independently verify evidence; interrupt and recover a worker; integrate verified work; perform integration verification; promote; reconstruct state from disk in a fresh session; then deliberately attempt forbidden transitions and stale-evidence reuse. Parallel execution should be enabled only after the single-EU path is reliable.

13. Staged Implementation Strategy

The architecture should be introduced around a still-working Codex installation rather than by deleting accumulated state. Existing global Codex material is first inventoried and treated as legacy. A clean project-local control environment is created beside it. Each stage begins from a known-good Git baseline, implements one narrow capability, runs positive and negative tests, undergoes independent verification, records evidence, and receives a known-good tag before the next stage.

Stage	Capability
A	Baseline and contamination audit
B	Clean project + Git + isolation
C	Durable state + append-only audit
D	Immutable intent / mutable graph
E	Role separation
F	Context Compiler v1 and exact path resolution
G	Scout and Context Requests
H	Project Map v1
I	Human Kanban gates and Obsidian views
J	Independent verification and evidence binding
K	Single-EU end-to-end proof

L	Recovery/resume
M	Integration and integration verification
N	Parallel execution
O	Replan/amendment machinery
P	Knowledge curation and retrieval
Q	Maintenance and break-glass proof

14. Discussion

14.1 Intelligence Versus Authority

The architecture deliberately refuses to equate model capability with system authority. A stronger model may plan better, code better, and diagnose failures better, yet those improvements do not logically require granting it the right to redefine requirements, choose its own proof, or promote its own work. This separation resembles established engineering practice in which design, implementation, testing, release, and governance can be performed by capable people while remaining institutionally distinct.

14.2 Rigidity Versus Adaptability

A deterministic control plane can become dangerous if it prevents correction of its own assumptions. The solution is not unrestricted agent discretion but layered authority: mutable execution graphs, explicit amendments, planned maintenance mode, and a human-only break-glass mechanism. Thus adaptability is preserved without turning every exception into an escape hatch.

14.3 Cost as an Architectural Variable

Token and tool cost are treated as design variables rather than billing afterthoughts. Exact context resolution, progressive disclosure, disposable fresh sessions, derived project maps, and telemetry-based retrieval policies all seek to minimize unnecessary model exposure while retaining enough information for reliable work. This makes context quality—not context volume—a target for optimization.

15. Limitations

This thesis describes an architecture under active development. Several components have stronger empirical support than others. Prime/IDD phase results demonstrate specific controls and specific failure modes but do not yet establish general reliability

across languages, repositories, organizations, or model families. The proposed Context Compiler, Project Map, visual supervision layer, and telemetry-driven policy learning require systematic evaluation. Filesystem isolation and capability enforcement will also vary by operating system and agent runtime. Finally, human approval remains a bottleneck by design during early trust-building; the appropriate threshold for reducing human gates must be learned empirically rather than assumed.

16. Future Work

- Implement the staged architecture around Codex while preserving its existing global installation as legacy state.
- Create a formal schema for intent, graph revisions, EUs, context packets, evidence, human approvals, and replans.
- Build the deterministic Project Map and measure its effect on search/tool cost.
- Compare context policies using first-pass success, token cost, and verifier failure.
- Develop controlled curation of historical Codex, Claude, and Hermes vaults.
- Quantify the optimal size and shape of vertical execution units from telemetry.
- Test selective removal of human gates only after evidence supports specific autonomy classes.
- Evaluate the architecture across different coding-agent runtimes to test framework neutrality.

17. Conclusion

Agentic software engineering changes the unit of engineering control. When a model can plan, edit, execute, test, and continue autonomously, reliability cannot rest solely on better prompting or a more capable model. The project needs durable intent, explicit authority, narrow context, independent proof, reconstructable state, and human-governed exception paths.

The architecture developed here combines IDD, TDD, LPVM, context engineering, deterministic lifecycle control, evidence-bound verification, and project-local knowledge into a coherent answer. Its defining move is to preserve intelligence inside agents while relocating authority outside them. Agents remain flexible, creative, and replaceable. The controller remains comparatively simple, deterministic, and accountable. Human intent remains the highest project authority.

The resulting research program is therefore not an attempt to make an AI agent perfectly obedient. It is an attempt to build an engineering system in which perfect obedience is unnecessary: the agent can reason freely inside a lane whose boundaries, evidence requirements, and legal transitions are controlled elsewhere.

References

1. Böckeler, B. (2026). “Context Engineering for Coding Agents.” MartinFowler.com. <https://martinfowler.com/articles/exploring-gen-ai/context-engineering-coding-agents.html>
2. Krishnan, H. (2026). “Intent-Driven Development.” intent-driven.dev. <https://intent-driven.dev/knowledge/intent-driven-development/>
3. Krishnan, H. (2026). “Spec-Driven Development.” intent-driven.dev. <https://intent-driven.dev/knowledge/spec-driven-development/>
4. Krishnan, H. (2026). Spec-Driven Development: Engineering with Intent. Manning (early access). <https://www.manning.com/books/spec-driven-development>
5. J-Tech Japan. (2026). “Intent-Driven Development.” <https://www.intent-driven-development.com/>
6. MCKRUZ. (2026). “Intent Driven Development — The Delivery Standard.” GitHub. <https://github.com/MCKRUZ/intent-driven-development>
7. Exadra37. (2026). “AI Intent Driven Development guidelines.” GitHub. <https://github.com/Exadra37/ai-coding-agents>
8. Pocock, M. (2026). “AI Coding Workflow.” AI Engineer. <https://ai.engineer/talks/-QFHioCo-Ko-ai-coding-workflow>
9. Beck, K. (2002). Test Driven Development: By Example. Addison-Wesley.

Appendix A. Proposed Core State Model

The following conceptual state model summarizes the principal lifecycle entities. Exact serialization should be selected during implementation and validated with schema tests.

Entity	Purpose
IntentVersion	Immutable approved destination and constraints
GraphVersion	Versioned decomposition and dependency state
ExecutionUnit	Small behavior-producing work contract
RoleExecution	One disposable worker invocation
ContextPacket	Immutable compiled context and capability envelope
EvidenceBundle	State-bound proof of execution and

	acceptance
HumanGate	Explicit controller-enforced approval requirement
ReplanRequest	Proposed change to route without changing destination
IntentAmendment	Human-approved change to destination
AuditEvent	Append-only lifecycle transition record
KnowledgeItem	Provenanced reusable project information
ProjectMap	Rebuildable derived relationships among project artifacts

Appendix B. Glossary

Term	Definition
IDD	Intent-Driven Development: preserving explicit human intent as the organizing authority for AI-assisted work.
TDD	Test-Driven Development: establishing tests/expected behavior before implementation; generalized here into evidence-first acceptance.
LPVM	Lean Production Vault Model: the project lineage emphasizing lean context, durable knowledge, reconstructability, and framework-neutral control.
EU	Execution Unit: a small, independently verifiable, preferably vertical unit of behavior.
Context Compiler	Controller-side component that assembles the smallest sufficient role-specific context packet.
Scout / Resolver	Search role used to resolve missing resources without turning execution workers into repository explorers.

Project Map	Derived structural index linking requirements, EUs, code, tests, evidence, and knowledge.
Promotion	Controlled transition of independently verified work into accepted canonical project state.
Break glass	Human-only emergency path for repairing a defective control plane when normal maintenance cannot proceed.